```
}

VOID Task1( VOID *args)
{
    RK_UNUSEARGS
    UINT write[2];
    while (1)
    {
        /* start sleeping */
        RK_TICK sleepTicks = ( RK_TICK) rand() % 3;
        if (sleepTicks == 0)
            sleepTicks = 3;

        kSleepUntil( sleepTicks);
        UINT currTick = kTickGet();
        UINT speedValue = ( UINT) rand() % 170;
        write[1] = currTick;
        write[0] = speedValue;
        /* grab a buffer */
        RK_CAB *bufPtr = (RK_CAB*) kCABReserve( &cabCtl);
        if (bufPtr != NULL)
        {
            /* copy the contents of write[] into bufPtr
            and publish it to readers */
            kCABPublish(&cabCtl, bufPtr, write);
        }
        else
        {/* cannot fail */
            kassert( 0);
        }
        kprintf( "! @ %u: SPEED UPDATE: %u \n\r", currTick, speedValue);

    }
}

void Task2( void *args)
{
    RK_UNUSEARGS
    UINT read[2];
    while (1)
    {
        SLEEP:
        /* pend on private semaphore */
        kTaskSemaPend(RK_WAIT_FOREVER);
        /* return the address of the most recent buffer
           and copy its contents to read  */
        RK_CAB *readBuf = kCABGet( &cabCtl, read);
        if (readBuf == NULL)
        {
            goto SLEEP;
        }
```

```
        /* use read[] to process the contents */
        kprintf( "@ %lu CRUISE - %u : %u \n\r", kTickGet(), read[0], read[1]);
        /* use the buffer address to release the buffer */
        kCABUnget( &cabCtl, readBuf);

    }
}

VOID Task3( VOID *args)
{
    RK_UNUSEARGS
    UINT read[2];
    while (1)
    {

        SLEEP:
        kTaskSemaPend(RK_WAIT_FOREVER);
        RK_CAB *readBuf = kCABGet( &cabCtl, read);
        if (readBuf == NULL)
        {
            goto SLEEP;
        }
        kprintf( "@ %lu WHIPERS - %u @ %u \n\r", kTickGet(), read[0], read[1]);
        kCABUnget( &cabCtl, readBuf); /* release buffer */

     }
}
VOID Task4( VOID *args)
{
    RK_UNUSEARGS
    UINT read[2];
    while (1)
    {
        SLEEP:
        kTaskSemaPend(RK_WAIT_FOREVER);
        RK_CAB *readBuf = kCABGet( &cabCtl, read);
        if (readBuf == NULL)
        {
            goto SLEEP;
        }
        kprintf( "@ %lu RADIO - %u @ %u \n\r", kTickGet(), read[0], read[1]);
        kCABUnget( &cabCtl, readBuf); /* release the buffer */

    }

}
```

What we expect to see is that sometimes all the 3 sensors get a new update, sometimes only one or two (ordered by priority) and sometimes all sensors miss an update. Importantly, they always get

the last updated value.

The picture below shows when an update happens ("!") and the new speed data - and when ("@") the tasks got the updated value followed by the time it was updated.

```
! @ 1328: SPEED UPDATE: 51
@ 1329 CRUISE - 51 : 1328
@ 1330 WHIPERS - 51 @ 1328
@ 1331 CRUISE - 51 : 1328
@ 1332 RADIO - 51 @ 1328
@ 1333 WHIPERS - 51 @ 1328
@ 1334 CRUISE - 51 : 1328
! @ 1335: SPEED UPDATE: 168
! @ 1337: SPEED UPDATE: 33
! @ 1338: SPEED UPDATE: 93
! @ 1339: SPEED UPDATE: 64
@ 1340 CRUISE - 64 : 1339
@ 1341 CRUISE - 64 : 1339
@ 1342 WHIPERS - 64 @ 1339
! @ 1343: SPEED UPDATE: 103
@ 1344 CRUISE - 103 : 1343
@ 1345 WHIPERS - 103 @ 1343
@ 1346 CRUISE - 103 : 1343
@ 1347 RADIO - 103 @ 1343
@ 1348 WHIPERS - 103 @ 1343
@ 1349 CRUISE - 103 : 1343
! @ 1350: SPEED UPDATE: 14
! @ 1351: SPEED UPDATE: 106
@ 1352 CRUISE - 106 : 1351
@ 1353 WHIPERS - 106 @ 1351
@ 1354 RADIO - 106 @ 1351
! @ 1355: SPEED UPDATE: 84
@ 1356 CRUISE - 84 : 1355
@ 1358 CRUISE - 84 : 1355
! @ 1359: SPEED UPDATE: 32
! @ 1360: SPEED UPDATE: 9
@ 1361 CRUISE - 9 : 1360
@ 1362 WHIPERS - 9 @ 1360
@ 1363 WHIPERS - 9 @ 1360
! @ 1364: SPEED UPDATE: 165
@ 1365 CRUISE - 165 : 1364
! @ 1366: SPEED UPDATE: 106
```

Design Patterns

Check www.kernel0.org/blog for updates on Design Patterns using ***RK0*** along with fundamental RTOS concepts.

© 2025 Antonio Giacomelli | All Rights Reserved | www.kernel0.org